

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **5**
Topic : **1**

Introduction to MonetDB

Objectives

This session will give the knowledge about

- Introduction to MonetDB
- Key Concepts,
- Column-store Architectures vs MonetDB
- Introduction to MonetDB SQL
- Installation and basic CRUD Operations

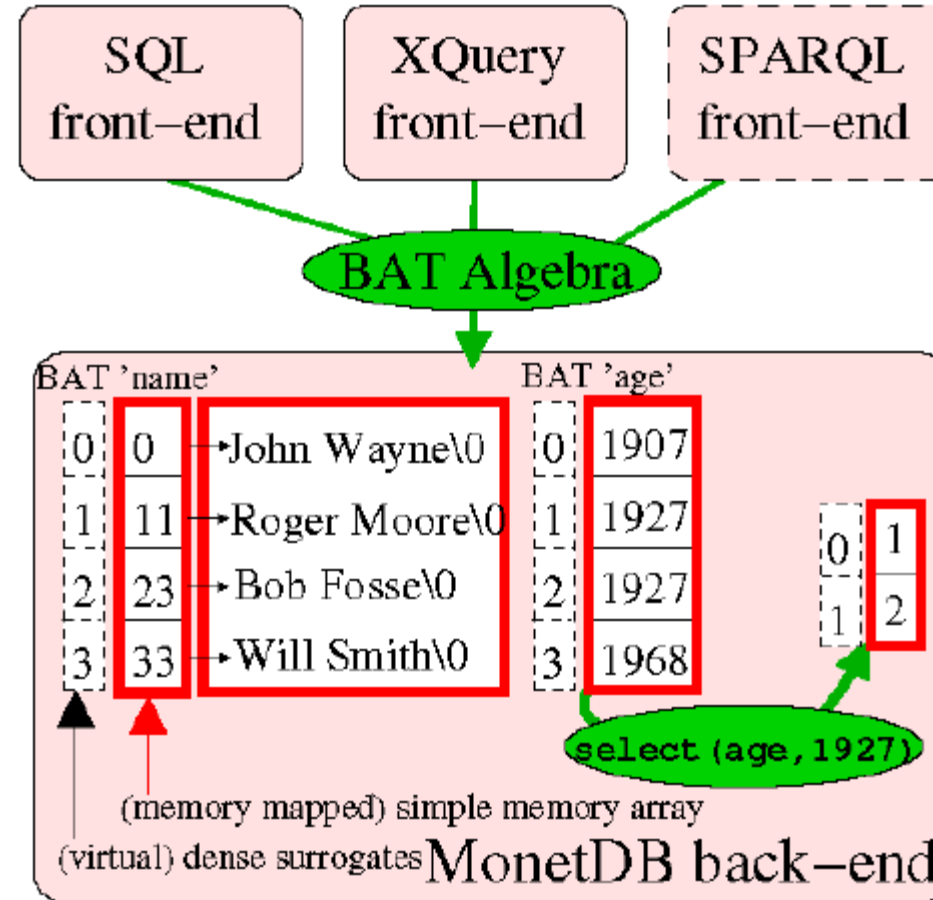
Introduction to MonetDB

- MonetDB is an **open-source, high-performance column-oriented database** designed for modern analytics and scientific data processing environments.
- It **stores data by columns instead of rows**, optimizing query execution speed, memory access, and hardware utilization for large datasets.
- First **introduced in 1993 by CWI**, MonetDB pioneered column-store innovations in the field of data warehousing and analytics.
- Widely used in business intelligence, eScience, and research contexts where efficient large-scale data analysis is required.
- MonetDB continues to evolve, offering advanced features, performance improvements, and strong community-driven development support.

Column-Store Database Kernel

- MonetDB represents database relations as columns (arrays) instead of rows, ensuring high efficiency in analytical workloads.
- Each column can scale to gigabytes, loaded dynamically into memory by the operating system as needed.
- This structure minimizes unnecessary data reads, improving speed for queries needing only selected attributes.
- Columnar representation allows for effective compression, reducing storage footprint and accelerating query performance significantly.
- MonetDB's architecture fundamentally differs from traditional row-store systems such as MySQL or PostgreSQL.

Column-Store Database Kernel



High-Performance Design

- MonetDB excels when datasets or “hot sets” largely fit within main memory for rapid access.
- Only necessary columns are accessed, reducing I/O and improving cache utilization for analytical queries.
- **Cache-conscious algorithms** further accelerate query execution, exploiting modern CPU cache hierarchies effectively.
- Suitable for workloads involving wide tables but queries touching only a few attributes per request.
- Offers competitive performance compared to traditional relational systems, especially in analytics and reporting domains.

Multi-Core Power Engine

- MonetDB is **optimized for parallel execution**, leveraging modern multi-core processors to reduce query response time.
- Supports complex query processing by distributing workload across multiple cores simultaneously.
- Incorporates techniques for distributed processing, but emphasizes thoughtful design instead of brute force scaling.
- **MapReduce suffices for simple parallel jobs**, while complex problems demand careful schema design and replication strategies.
- MonetDB showcases scalable performance improvements for real-world analytical workloads on multi-core hardware.

Versatile Algebraic Kernel

- MonetDB employs **MonetDB Assembly Language (MAL)**, an internal algebraic language for query execution.
- MAL bridges declarative SQL queries and low-level kernel operations efficiently.
- It enables flexible query translation pipelines, supporting optimization and parallelization strategies effectively.
- **MAL also supports distributed execution protocols** for multi-server deployments of MonetDB.
- **SQL-to-MAL compiler** ensures compliance with SQL standards while exploiting MonetDB's algebraic processing strengths.

Scalability and Size Limits

- MonetDB database size depends **heavily on system architecture and file system**, with 64-bit platforms supporting substantially larger capacities than 32-bit ones.
- **Each column is represented as a file**, where overall storage limitations are imposed by underlying operating system capabilities and available physical hardware resources.
- Practically, **MonetDB allows creation of tables with a very large number of columns**, enabling storage of extremely wide relational datasets.
- **User concurrency is configurable**, with system threads dynamically adapting resource allocation to match workload intensity and concurrent analytical processing requirements.
- **MonetDB's architecture supports horizontal and vertical scaling strategies**, making it suitable for small-scale teaching scenarios and enterprise-level big data analytics.

Extensibility and Customization

- MonetDB provides architectural extensibility at every layer, allowing developers and researchers to embed domain-specific features directly into the database kernel.
- **Query optimizer includes customizable hooks** where additional rules and strategies can be injected to adapt execution behavior for specific domain workloads.
- **Kernel supports custom bulk operators**, enabling implementation of high-performance computational primitives for specialized scientific and industrial application domains.
- MonetDB **integrates seamlessly with external scientific libraries**, extending database operations with advanced functions like matrix algebra, geospatial processing, and text analytics.

Broad Application Scope

- MonetDB includes **built-in support for external libraries** such as PCRE, Raptor, libxml, and GEOS to enable diverse analytic functions.
- **Data vault functionality** allows encapsulation of **external file formats**, bridging traditional database processing with legacy scientific or enterprise file-based workflows.
- Extensively used across genomics, astronomy, and environmental sciences, MonetDB demonstrates **versatility for both structured and semi-structured analytical data processing**.
- MonetDB adapts well for **heterogeneous data integration**, where relational records, time-series logs, and XML or RDF documents coexist within one environment.
- Its versatility enables MonetDB to **support business intelligence, eScience, and government applications** requiring robust high-performance analytical infrastructures.

SQL and Programming Interfaces

- MonetDB follows SQL:2003 standards, supporting keys, joins, triggers, views, and procedures, providing familiarity for students and industry professionals.
- Transactions are ACID-compliant, maintaining reliability and consistency even under concurrent workloads with multiple analytical and reporting queries executing simultaneously.
- Connectivity interfaces include JDBC, ODBC, PHP, Python, Ruby on Rails, and C/C++, supporting diverse application development environments and ecosystems.
- MonetDB allows developers to extend functionality using User-Defined Functions (UDFs) written in C, Python, or R programming languages. Broad programming interface support ensures MonetDB integration with modern application stacks, enhancing adoption across enterprise, academic, and open-source communities.

Open Source and Community

- MonetDB is released under Mozilla Public License, allowing free usage, modification, and distribution, encouraging educational and commercial adoption globally.
- Strong developer and user community actively contribute bug fixes, performance improvements, feature requests, and new innovative extensions continuously.
- Community support channels include documentation, tutorials, forums, mailing lists, and GitHub repositories, providing help for students and professionals worldwide.
- MonetDB Solutions company offers commercial services, consultancy, and enterprise-grade support, extending usability for mission-critical deployments.
- Open-source model ensures accessibility to students, enabling universities to integrate MonetDB into curricula without expensive proprietary database licensing costs

Oracle vs MonetDB

Aspect	Oracle RDBMS	MonetDB (Columnar DB)
Data Model	Oracle is a traditional row-oriented relational database , optimized for OLTP systems where fast inserts, updates, and transactions are required.	MonetDB is a column-oriented analytical database , optimized for OLAP queries, where reading specific columns from very large datasets is most important.
Query Performance	Oracle performs strongly in transactional queries with mixed read-write operations but can become slower for large analytical aggregations.	MonetDB provides exceptional performance for analytical queries like aggregates, filters, and joins , minimizing unnecessary data reads significantly.

Oracle vs MonetDB

Aspect	Oracle RDBMS	MonetDB (Columnar DB)
Storage Architecture	Oracle stores entire rows together , which is efficient for transactions needing multiple attributes but less efficient for analytics workloads.	MonetDB stores data column-by-column, enabling compression, faster scanning, and efficient memory access for analytics with fewer columns accessed.
Scalability and Usage	Oracle is widely deployed in enterprise applications, financial services, ERP , and mission-critical systems requiring robust transactional support.	MonetDB is primarily used in research, scientific computing, and business intelligence scenarios that require analyzing very large data volumes.
Licensing and Cost	Oracle is commercial , requiring expensive licensing, support, and maintenance fees, which can limit accessibility for smaller organizations.	MonetDB is open-source under MPL , free to use, accessible to students and researchers, encouraging experimentation without licensing restrictions.

Applications of MonetDB in Practice

- SciLens Project – Scientific portal using MonetDB and SciQL for large-scale database processing of scientific applications and complex array queries.
- COMMIT/ – National ICT project (Netherlands) using MonetDB for real-time harvesting of spatiotemporal data from people, sensors, and observatories.
- LDBC – European initiative building benchmarks for RDF data management systems, with MonetDB contributing to Semantic Web data handling.
- ACTiCLOUD – Cloud project improving scalability and utilization of cloud resources, leveraging MonetDB for holistic distributed data management.
- FashionBrain – Industry project using MonetDB to predict fashion trends from social media and provide personalized recommendations to customers.

Applications of MonetDB in Practice

- ExaNeSt – HPC project supporting exascale supercomputing, where MonetDB helps with interconnection networks, storage management, and performance optimization.
- LOD2 Project – Focused on scalable management of Linked Data collections, integrating MonetDB to manage billions of RDF triples efficiently.
- Earth Observation Infrastructure – MonetDB used for remote sensing data analytics, supporting forest fire detection and management applications.
- EMILI Project – Emergency management project using MonetDB's event-stream processing for SCADA systems in metro stations and airport calamities.
- LEO Project (Precision Farming) – MonetDB supports linked open Earth Observation data for agriculture, enabling precision farming applications in Europe.

Columnar Database Comparison

Feature / Aspect	MonetDB	Amazon Redshift	Apache Cassandra
Type	Open-source columnar RDBMS optimized for analytics and scientific workloads.	Cloud-based, fully managed columnar data warehouse service by AWS.	Distributed, wide-column NoSQL store optimized for scalability and availability.
Primary Use Case	Analytical queries, BI dashboards, scientific data, research projects, and academic teaching.	Enterprise-scale analytics, reporting, ETL pipelines, and large-scale business intelligence.	High-volume real-time applications like IoT, logging, and time-series workloads.
Deployment	On-premise or self-hosted on Linux; easy academic/research usage.	Fully cloud-managed service tightly integrated with AWS ecosystem.	Distributed cluster deployment across on-premises or hybrid cloud setups.

Columnar Database Comparison

Feature / Aspect	MonetDB	Amazon Redshift	Apache Cassandra
Query Language	SQL:2003 compliant with extensions (SciQL for scientific arrays).	ANSI SQL with extensions for analytics and data warehousing.	CQL (Cassandra Query Language), SQL-like but limited joins/aggregates.
Performance Strengths	Excellent for columnar scans, aggregates, joins, and ad hoc analytical queries.	Optimized for petabyte-scale analytics with parallel query execution.	Very high write throughput, fault tolerance, and linear horizontal scalability.
Transactions	Fully ACID-compliant, suitable for reliable analytics workloads.	ACID for transactions but tuned more for read-heavy analytics workloads.	Offers tunable consistency, but not full ACID compliance, prioritizing availability.

Columnar Database Comparison

Feature / Aspect	MonetDB	Amazon Redshift	Apache Cassandra
Cost / Licensing	Free, open-source (Mozilla Public License).	Pay-as-you-go pricing, based on storage and compute usage.	Open-source under Apache License, with commercial support available (DataStax).
Extensibility	Integrates with scientific libraries, supports UDFs in C, Python, and R.	Extensible through AWS ecosystem services (S3, Glue, SageMaker).	Supports custom data models and replication strategies for diverse workloads.

MonetDB SQL manual

- **SQL and standards:** SQL evolved through multiple standards up to SQL:2016; real systems rarely implement everything due to legacy, compatibility, and new features.
- **MonetDB SQL lineage:** MonetDB's SQL front-end began in 2002, grounded in SQL'99 and later SQL'03; less-used features trail, non-standard extensions are explicitly marked.
- **Execution architecture:** MonetDB compiles SQL into MAL (MonetDB Assembly Language), applying relational optimizations; bindings exist for C/ODBC/Mapi, Java/JDBC, Python, PHP, and Ruby on Rails.
- **Syntax notation:** Features use simplified BNF: CAPITALS for reserved words; [] optional parts; { } choose one; | alternatives; lists show repetition; identifiers may carry semantic tags.

MonetDB SQL – Lexical Structure: Comments

- MonetDB SQL supports two comment styles: double-dash -- until line end, and C-style /* ... */ inline annotations. Comments can be added anywhere a token is valid, helping improve readability and documentation within SQL query scripts.

/ Insert sample employees */*

INSERT INTO emp (eid -- Employee ID, ename, salary) VALUES (1, 'Aarav Mehta', 65000.00);

- White space (spaces, tabs, newlines) separates tokens unless juxtaposition remains unambiguous, ensuring correct parsing and interpretation.
- Comments are equivalent to white space, ignored by the parser, and have no effect on SQL query execution semantics.

Identifiers and Keywords

- SQL keywords are reserved by the standard, while **MonetDB distinguishes keywords, identifiers, and user-defined names during query parsing.**
- **Unquoted identifiers are case-insensitive**, automatically converted to lowercase, while quoted identifiers preserve case sensitivity exactly as written.
- For example, select TEST becomes select test, while select "TEST" preserves uppercase meaning for a column or object.
- Double quotes allow identifiers with spaces, special characters, or reserved keywords, useful for user-friendly column headers or compatibility.
- **Restrictions:** identifiers cannot start with %, cannot equal TID%, and cannot exceed 1024 characters in total length.

Literal Constants

- **Numeric constants** follow typical programming syntax, with integers, decimals, and floating-point values inferred from presence of dots or exponents.
- **String constants** use single quotes, doubling internal quotes for escapes, e.g., 'a single " quote' results in proper embedded quote.
- **Unicode strings** use U&'...' UESCAPE, allowing multilingual data support and explicit escape definitions within SQL queries.
- **PostgreSQL-style** E'...' strings enable C-style escape sequences, e.g., \n for newline, enhancing flexibility in string representation.
- **Raw strings** use R'...', disabling escape processing, preserving literal characters exactly as typed, useful for regex patterns.

Temporal and Binary Constants

- **Temporal literals:** DATE 'YYYY-MM-DD', TIME 'HH:MM:SS', TIMESTAMP 'YYYY-MM-DD HH:MM:SS' map to respective MonetDB SQL temporal types.
- Equivalent **type casting is possible** with **CAST('2014-02-03' AS DATE)** ensuring explicit type control across queries.
- **Binary values are represented as hexadecimal**, e.g., x'10FF' for 2-byte values, supporting binary large object (BLOB) manipulation.
- Strings and constants can be explicitly cast into integers, decimals, or floating-point using **CAST(expr AS type)** operations.
- Casting ensures compatibility between types, enabling numeric operations on strings or format-specific conversions during query processing.

Special Characters

- **String literals** support escape sequences: `\n` **newline**, `\t` **tab**, `\r` carriage return, `\f` form feed, and `\\` backslash.
- Escapes also include `\'` **for quotes**, octal codes `\ddd`, hexadecimal `\xhh`, and **Unicode** `\uhhhh` for specific code points.
- C-style backslash escapes provide rich character encoding flexibility, consistent with programming language conventions.
- These escapes improve interoperability, ensuring text data with control characters or non-standard symbols is represented accurately.
- Essential for queries handling text data, logs, or cross-platform string formats within MonetDB applications.

Operator Precedence

- SQL operators have defined precedence and associativity, ensuring predictable evaluation **order of arithmetic, logical, and comparison expressions**.
- Parentheses disambiguate precedence, allowing explicit grouping of operations when complex expressions require clarity in evaluation order.
- Operators include arithmetic (+, -, *, /), logical (AND, OR, NOT), comparison (=, <, >), and pattern-matching (LIKE).
- MonetDB disallows overloading operators or defining custom ones, maintaining consistency with SQL standards for predictable semantics.
- Operator precedence supports intuitive expression evaluation but requires awareness to avoid subtle errors in query conditions.

Value Expressions

- Value expressions include constants, type casts, column references, operator expressions, function invocations, and scalar-returning subqueries.
- Column references may use table.column or aliases defined in FROM clauses, ensuring clarity when multiple tables are involved.
- Table expressions reference schema, table, or view names, optionally aliased with AS to simplify query readability and correlation.
- Expressions may combine multiple constructs, supporting nested functions, complex conditions, and multi-level subqueries.
- This flexible structure enables MonetDB SQL to express sophisticated analytics queries within standard relational syntax.

Introduction to Data Types

- SQL defines many built-in types; MonetDB supports most, ensuring compliance while enabling strong foundations for relational database management systems.
- MonetDB allows creation of user-defined types, supporting specific application domains needing specialized semantics beyond built-in SQL data types.
- Some domain-specific types, such as geospatial data, are available as separate extensions, not part of MonetDB's database kernel.
- Extensions like spatial types broaden MonetDB's capabilities, making it suitable for advanced domains like GIS, environment, and mapping.
- MonetDB balances standard SQL compliance with flexibility for custom application-driven data type support.

Base Types

- MonetDB provides core base types: numeric (INT, BIGINT, DECIMAL), string (CHAR, VARCHAR, CLOB), and binary (BLOB, BYTE).
- INTEGER types support varying ranges; BIGINT for very large numbers, while SMALLINT and TINYINT handle smaller integer values.
- DECIMAL and NUMERIC allow fixed-precision arithmetic, essential for financial, scientific, and industrial applications requiring exact calculations.
- VARCHAR supports variable-length text up to defined limits, CHAR ensures fixed-length storage, and CLOB handles very large text.
- BLOB and BYTE manage binary data, such as multimedia files or encoded streams, with flexible storage and retrieval capabilities.

Temporal Types

- MonetDB supports DATE, TIME, TIMESTAMP, and INTERVAL, enabling detailed representation of temporal information in relational queries.
- DATE uses format YYYY-MM-DD, TIME records hours, minutes, and seconds, while TIMESTAMP combines both into one field.
- INTERVAL represents differences between two temporal values, supporting arithmetic like adding months, days, or hours to timestamps.
- CAST operations allow conversion between string formats and temporal values, ensuring compatibility across queries.
- Precise temporal typing supports applications like event logging, scheduling, time-series analytics, and scientific experiments.

Serial Types

- MonetDB provides SERIAL, BIGSERIAL, and related auto-incrementing types for generating unique identifiers automatically during insert operations.
- SERIAL expands into INTEGER with a default NEXT VALUE FOR sequence, ensuring compliance with SQL semantics.
- BIGSERIAL maps to BIGINT with sequence-driven increments, suitable for datasets requiring billions of unique identifiers.
- These types simplify schema design by eliminating manual sequence creation, supporting primary keys directly within tables.
- SERIAL usage improves readability and consistency, particularly in OLTP and analytical scenarios requiring surrogate keys

JSON Datatype

- MonetDB includes JSON data type, enabling storage and manipulation of JSON documents directly inside relational schema.
- Supports validation of JSON structure, ensuring data integrity during inserts and updates into JSON-typed columns.
- SQL functions allow extraction and querying of fields within JSON documents, bridging relational and semi-structured data paradigms.
- Applications include web logs, API payloads, configuration data, and hybrid systems requiring flexible schema-less storage.
- JSON integration improves MonetDB's versatility for modern applications blending structured and unstructured data.

URL Datatype

- MonetDB offers URL type for managing resource locators within relational schema.
- URL type ensures consistent representation of web addresses and network resources across queries.
- Useful for web analytics, metadata catalogs, or content management systems linking to external resources.
- MonetDB enforces structural validity on URLs, preventing malformed addresses from corrupting datasets.
- URL data type simplifies working with internet-based datasets inside MonetDB applications.

UUID Datatype

- MonetDB supports UUID type for representing Universally Unique Identifiers across distributed systems and applications.
- UUID ensures globally unique keys independent of central sequence generators.
- Suitable for IoT, cloud, and distributed computing applications needing decentralized key generation.
- UUID columns integrate smoothly into relational queries while maintaining uniqueness guarantees.
- This datatype strengthens MonetDB's applicability in enterprise-grade and multi-tenant environments.

Network Datatypes

- MonetDB supports IP4, IP6, and MAC address types under Network category.
- These ensure accurate representation of networking information, avoiding storage as plain strings.
- Useful in logging, network monitoring, and security analytics.
- Network types can be queried and validated using built-in functions.
- Integration simplifies tasks in IoT, telecom, and system monitoring contexts

User Defined Datatypes

- MonetDB allows developers to create user-defined types tailored to domain-specific applications.
- Enables encapsulation of specialized semantics like chemical structures, genomic codes, or sensor data.
- UDTs extend beyond built-in SQL types, offering flexibility and innovation in research projects.
- Support for UDTs highlights MonetDB's experimental and extensible architecture.
- Useful in advanced analytics, bioinformatics, and niche industries requiring custom data modeling

MonetDB Installation

- Download: <https://www.monetdb.org/easy-setup/>
- Install with default values.
- Run Server from start menu
- Run Client and execute all queries

Datatypes Example

```
CREATE TABLE emp_reg (  
  id      INT,           -- integer  
  age     SMALLINT,      -- small integer  
  salary  DECIMAL(5,2),   -- fixed precision decimal  
  initial CHAR(1),        -- single character  
  name    VARCHAR(50),    -- variable length string  
  dt      DATE,           -- calendar date  
  tm      TIME,           -- clock time  
  profile JSON,           -- JSON document  
  ip      INET,           -- IP address  
  sid     SERIAL,         -- auto increment ID  
  link    URL,            -- URL link  
  uid     UUID            -- universally unique identifier  
);
```

Datatypes Example

```
INSERT INTO emp_reg (  
  id, age, salary, initial, name, dt, tm, profile, ip, link, uid  
)  
VALUES (  
  1,                -- INT  
  25,               -- SMALLINT  
  55000.75,         -- DECIMAL(5,2)  
  'A',             -- CHAR(1)  
  'Arjun Patel',    -- VARCHAR  
  DATE '2000-05-15', -- DATE  
  TIME '09:30:00',  -- TIME  
  '{"skills":["SQL","Python"],"exp":3}', -- JSON  
  '192.168.1.10',   -- INET  
  'https://www.example.com/profile/arjun', -- URL  
  SYS.UUID()        -- UUID (auto-generated unique ID)  
);
```

Basic CRUD Operations

Create and use schema

```
CREATE SCHEMA IF NOT EXISTS company;
```

```
SET SCHEMA company;
```

Create two users (requires admin privileges)

```
CREATE USER hr WITH PASSWORD 'hr@123' NAME 'HR User' SCHEMA  
company;
```

```
CREATE USER employee WITH PASSWORD 'employee@123' NAME 'Employee  
User' SCHEMA company;
```

Basic CRUD Operations

Create Table:

```
CREATE TABLE emp ( eid SERIAL, ename VARCHAR(100) NOT NULL, salary  
DECIMAL(10,2) NOT NULL, skills VARCHAR(100) );
```

```
CREATE TABLE emp1 ( eid INT PRIMARY KEY AUTO_INCREMENT, ename  
VARCHAR(100) NOT NULL, salary DECIMAL(10,2) NOT NULL, skills  
VARCHAR(100) );
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON company.emp TO hr;
```

```
GRANT SELECT ON company.emp TO employee;
```

Basic CRUD Operations

Insert:

```
INSERT INTO emp (ename, salary, skills) VALUES('Aarav Mehta', 65000.00,  
'SQL,Python,ETL'),('Meera Iyer', 72000.50, 'Java,Spring,APIs'),('Rahul  
Sharma',59000.00, 'Excel,PowerBI');
```

Retrieve:

```
SELECT * FROM emp;
```

```
SELECT * FROM emp WHERE salary >= 60000.00;
```

```
SELECT ename FROM emp WHERE skills LIKE '%Python%';
```

Basic CRUD Operations

Update:

```
UPDATE emp SET salary = ROUND(salary * 1.10, 2) WHERE ename = 'Meera Iyer';
```

```
SELECT * FROM emp;
```

Delete:

```
DELETE FROM emp WHERE ename = 'Rahul Sharma';
```

```
ALTER TABLE emp DROP COLUMN skills;
```

Basic CRUD Operations

DROP TABLE emp;

SET SCHEMA sys; -- switch schema first

DROP SCHEMA company CASCADE;

Summary

This session will give the knowledge about

- Introduction to MonetDB
- Key Concepts,
- Column-store Architectures vs MonetDB
- Introduction to MonetDB SQL
- Installation and basic CRUD Operations